NP completeness

PSPACE: decision problems solvable in polynomial space **Co-NP:** complements of decision problems in NP **NP:** …… there is a poly-time algorithm certifier **P(EXP):** ……for which there is a poly(exponential)-time algorithm

Reduction $X \le_p Y$ if arbitrary instances of problem X can be solved using 用 X 实例构造 Y

• Polynomial number of standard computational steps, plus • Polynomial number of calls to oracle that solves problem Y

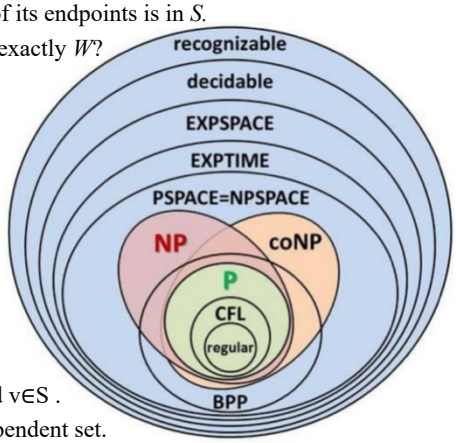
NP-complete: A problem Y in NP with the property that for every problem X in NP, $X \le_p Y$

Independent set: Given a graph G = (V, E) and an integer k, is there a subset of vertices $S \subseteq V$ such that $|S| \ge k$, and for each edge **at most** one of its endpoints is in S. **Vertex cover:** ⋯ such that $|S| \le k$, and for each edge **at least** one of its endpoints is in S.

Subset-sum: Given natural numbers $w_1, w_2, \ldots, w_n$ and an integer W, is there a subset that adds up to exactly W?

Set cover: Given a set U of elements, a collection $S_1, S_2, \ldots, S_m$,of subsets of U, and an integer k, does there exist a collection of $\le k$ of these sets whose union is equal to U

CIRCUIT-SAT: Given a combinational circuit built out of AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?



3-SAT: Literal: x_i或$\neg x_i$。**Clause:** （∨）。**CNF:** 由子句的合取（∧）构成的命题公式。

SAT：Given CNF formula ϕ, does it have a satisfying truth assignment?

Recipe Y is NPC Y is in NP → Choose an NP-complete problem X→ Prove that $X \le_p Y$

Vertex-cover $\equiv_p$Independent-set Pf. We show S is an independent set if V-S is a vertex cover → Let S be any independent set. Consider an arbitrary edge (u,v). S independent→ $u \notin$ S or $v \notin$ S→ u∈V-S or v∈V-S Thus,V-S covers (u,v) ← Let V -S be any vertex cover ·Consider two nodes u∈S and v∈S . Observe that (u,v) ∉ E since V -S is a vertex cover Thus, no two nodes in S are joined by an edge S independent set.

3-SAT is NPC Suffices to show that CIRCUIT-SAT ≤p 3-SAT since 3-SAT is in NP.

Let K be any circuit. We can construct an instance of 3-SAT. Step 1: Create a 3-SAT variable $x_1$for each circuit element i. Step 2: Make circuit compute correct values at each node. A) $x_2 = \neg x_3$ → add 2 clauses: $x_2 \vee x_3$, $\overline{x_2} \vee \overline{x_3}$ B) $x_1 = x_4 \vee x_5$ → add 3 clauses: $x_1 \vee \overline{x_4}$, $x_1 \vee \overline{x_5}$, $\overline{x_1} \vee x_4 \vee x_5$ C) $x_0 = x_2 \wedge x_3$→ add 3 clauses: $\overline{x_0} \vee x_1$, $\overline{x_0} \vee x_2$, $x_0 \vee \overline{x_1} \vee \overline{x_2}$ Step 3: Hard-coded input values and output value x_5= 0 → add 1 clauses: $\overline{x_5}$, x_0= 1 →add 1 clauses: x_0 Final step: turn clauses of length < 3 into clauses of length exactly 3

• Lemma. **ϕ** is satisfiable iff the inputs of K can be set s.t. it outputs 1 ←Suppose there are inputs of K s.t. it outputs 1 • Compute the values of all the nodes in K • According to our construction, this set of values satisfies. → Suppose **ϕ** is satisfiable • The variable assignment of **ϕ** specifies the values of all the nodes in K • Our construction ensures that all the nodes in K are correctly computed and the output is 1

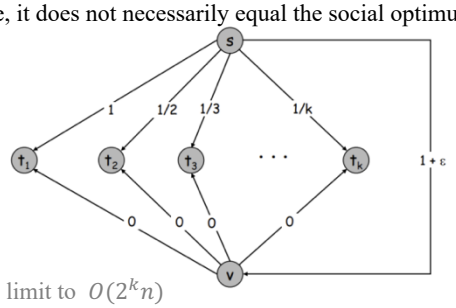
Nash Equilibria Social optimum: Minimizes total cost to all agents. Price of stability: Ratio of best Nash equilibrium to social optimum.

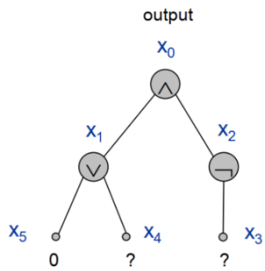
Observation: there may be many Nash equilibria. Even if it's unique, it does not necessarily equal the social optimum.

Ex: Price of stability = Θ(log k).

- **Social optimum:** Everyone takes bottom paths.
- **Unique Nash equilibrium:** Everyone takes top paths.
- **Price of stability:** H(k) / (1 + ε).

1 + 1/2 + … + 1/k





FPT Brute force 难以求解，顶点覆盖问题 $O(kn^{k+1})$=$n^k \cdot nk$ limit to $O(2^k n)$

fixed parameter tractable with respect to some parameter k is the class of problems solvable in time $f(k) \cdot poly(n)$.

Circular Arc Covering brute force:$O(k^m)$ 现有算法 $O(k! \cdot n)$ m 路径数，k 着色数, n 节点数 This is poly(n) time if k =lgn/lglg n 想$O(k!\,n) \leftrightarrow poly(n)$ ∵ $k! = O(k^k)$ →想 $O(k^k) = O(n^c)$取对数 $O(k\log k) = O(c\log n)$ 取对数 $O(\log k + \log\log k) = O(\log\log n)$ 除

Amortized Analysis

Stack operations: PUSH(S, x)→ $O(1)$, POP(S) → $O(1)$, MULTIPOP(S, k) →$O(min\,|S|, k)$

Aggregate Analysis calculates 操作总成本 and amortizes it evenly across each operation to derive the amortized cost.

1.For n_{push} PUSH operations, each has a cost of $O(1)$，so the total cost is $O(n_{push})$ 2. ……3. For n_{MULTIPOP} MULTIPOP operations: although the actual cost of each operation is $O(min\,|S|, k)$, the total number of elements popped by these operations cannot exceed the number of elements pushed by previous PUSH operations. Thus, the total cost is still bounded by n_{push}.→ Since the total number of operations =n_{push}+n_{pop}+n_{MULTIPOP} ≤2*n_{push}, the total cost is $O(n_{push})$= O(n), ,and the amortized cost per operation is O(n)/n= $O(1)$

Potential Method $c_i' = c_i + \phi(D_i) - \phi(D_{i-1})$

for a stack := the number of objects in the stack. So $\phi(D_0)$=0, and $\phi(D_i)$ ≥0

PUSH: Potential change: $\phi(D_i) - \phi(D_{i-1})$= 1. Amortized cost:$c_i' = c_i + \phi(D_i) - \phi(D_{i-1})$ = 1 + 1 = 2. POP: ⋯= -1. Amortized cost: ⋯= 1 + (-1) = 0. MULTIPOP(S, k): ⋯ –k. Amortized cost: ⋯= 0. So amortized cost of each operation is O(1)

Dynamic Table: Aggregate analysis: The cost of insertion O(1);The cost of rebuild: $O(m)$, m is current size of table. Total cost of n insertions themselves: O(n)；Total cost of all rebuilds:1 + 2 + 4 + 8 + … + n ,copy log2n times =2n= O(n). Amortized cost = 3n/n=3O(1)

Define the potential of the table after the i th insertion by $\phi(D_i)$=$2n_i - m_i$, n 是数组中的元素数量，m 是数组的当前容量, So $\phi(D_0) = 0$, and $\phi(D_i)$. 插入操作（无需扩容）：操作成本：$O(1)$,势能变化：$\phi(D_i) - \phi(D_{i-1})$ =$(2n_i - m_i) - (2(n_i - 1) - m_i)$=2 均摊成本：1 +2 =3. 插入操作（触发扩容）操作成本：$O(n)$,因为需要将所有元素复制到新数组中，并插入新元素．势能变化：扩容后，容量增加，势能减少，变化大小为 $\phi(D_i) - \phi(D_{i-1})$ =$(2n_i - 2(n_i - 1)) - (2(n_i - 1) - (n_i - 1))$=2-$(n_i - 1)$=3-$n_i$ 均摊成本：n_i +3-n_i=3

观察：不同位的翻转频率不同——第i位（从0开始计数）每2^i次 INCREMENT 翻转1次；总翻转次数计算：n次操作中，第i位的翻转次数为$\lfloor n/2^i \rfloor$，总翻转次数为$\sum_{i=0}^{\log n-1} \lfloor n/2^i \rfloor$；级数求和：该级数为等比级数，和为≤ $2n$（即O(n)）；平摊成本：每个 INCREMENT 的平摊成本为$O(n)/n = O(1)$。

Approximation Algorithms

Approximation Ratio: If X is a maximization problem, A is an a-approximation algorithm for X if $A \geq OPT/a$ (a>1)
If X is a minimization problem, A is an a-approximation algorithm for X if $A \leq a \cdot OPT$ (a>1)

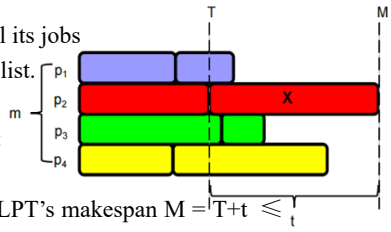
Set cover: A collection F of sets. $S \in F$, cost(S), $\bigcup_{S\in F} S = X$, choose $\bigcup_{S\in G} S = X, G \subseteq F$, make $min\ Cost(S \subseteq G)$

Greedy approximation algorithm: $U = X$, C = ∅; choose $S \in F - C$ with min $|cost\ (S)|/|S\cap U|$; $C = C\cup\{S\}$ $U = U - S$

Proof of Correctness: We always output a set cover, because the while loop continues **till** X is covered.

Proof of Approximation Ratio: $L = \sum_k \text{cost}\,(e_k) = \sum_k OPT/(n-k+1) \approx \ln(n) * OPT$

Makespan Scheduling (LS) n independent jobs. m processors. Assign to minimize the last processor finishes all its jobs



1. List the jobs in any order. 只要有 unfinished jobs. If any processor 没 a job now, give it the next job in the list.
3. Suppose LS gives makespan of M. optimal schedule have makespan M*. We prove that M ≤ 2M*
M* ≥ t, M* ≥ T (length of opt schedule is ≥ (total work)/m ≥ mT/m = T), M* ≥ max(T,t). M = T + t

LPT Scheduling: 1. Let's try to schedule longest jobs first. 3. Let X be the last job to finish.
Assume it starts at time T and has size t. Assume that X is the last job to start. → a）X is the smallest job. 1 LPT's makespan M = T+t ≤ M*+t Case 1 t ≤ M*/3. M ≤ M* + t ≤4/3 M*. Case 2 t > M*/3. all tasks have size > M*/3. the opt has at most 2 tasks per processor. So n≤2m. 1≤ n≤ m, both put one task per processor, m < n≤ 2m… LPT is optimal. →M ≤ 4/3 M*.

The Knapsack Problem: (PTAS) we want $\sum_{i\in S^*} v_i \leq (1+\varepsilon)\sum_{i\in S} v_i$

- $\sum_{i\in S^*} v_i \leq \sum_{i\in S^*} u_i \leq \sum_{i\in S} u_i \leq \sum_{i\in S}(v_i+\theta) \leq \sum_{i\in S} v_i + n\theta$

过渡 $\leq \sum_{i\in S} v_i + \varepsilon\sum_{i\in S} v_i = (1+\varepsilon)\sum_{i\in S} v_i$

- $n\theta = \varepsilon v^*/2 = \varepsilon v_j/2 \leq \varepsilon u_j/2 \leq \varepsilon/2 \sum_{i\in S} u_i$
- $\sum_{i\in S} v_i \geq \sum_{i\in S} u_i - n\theta \geq (2/\varepsilon - 1)n\theta$; $\epsilon \leq 1$, $n\theta \leq \epsilon \sum_{i\in S} v_i$

original S^*	scaled rounded problem	rounded S
v_i(=12500)	$\theta = \frac{\varepsilon v^*}{2n}, v_i' = \lceil \frac{v_i}{\theta} \rceil$(=13)	$u_i = v_i' \cdot \theta$(=13000)
	$v'^* = \lceil \frac{v^*}{\theta} \rceil = \lceil \frac{2n}{\varepsilon} \rceil, O(n^2 v'^*) = O(\frac{n^3}{\varepsilon})$	$v_i \leq u_i \leq v_i + \theta$

Vertex Cover: 1. D = {E}, C = ∅; Repeat as long as there are edge left in D{ Take any edge (u,v) in D; Add {u,v} to C.;Remove all the edges adjacent to u or v from D}; Output C. 2. We keep adding vertices till all edges are covered.
3. a) None of the edges in A touch each other. B) each vertex in C* covers at most one edge in A.→ |C*| ≥ |A|.
The number of vertices the algorithm uses is 2|A|.→ |C|/|C*| = 2|A| / |C*| ≤ 2|A| / |A| = 2.

Pricing Method WVC: $\sum_{e=(i,j)} p_e \leq w_i$(for vi), $\sum_{e\in E} p_e = \sum_{e\in E}\sum_{i\in S, e=(i,j)} p_e \leq \sum_{i\in S}\sum_{e=(i,j)} p_e \leq \sum_{i\in S} w_i = w(S)$总价格不超 V 总权重
1. p_e= 0; while (∃edge i-j such that neither i nor j are tight) select such an edge e, increase p_e without violating fairness) S ←set of all tight nodes 2. S is a vertex cover: if some edge i-j is uncovered, then neither i nor j is tight. But then while loop would not terminate.
3.$w(S) = \sum_{i\in S} w_i = \sum_{i\in S}\sum_{e=(i,j)} p_e \leq \sum_{i\in V}\sum_{e=(i,j)} p_e = 2\sum_{e\in E} p_e \leq 2w(S^*)$

LP Rounding(ILP->LP): 2. Consider an edge (i, j) ∈ E. Since $x_i^*+x_j^* \geq 1$, either $x_i^* \geq 1/2$ or $x_j^* \geq 1/2$ →(i, j) covered.
3. Let S* be optimal vertex cover. $\sum_{i\in S^*} w_i \geq \sum_{i\in S} w_i x_i^* \geq 1/2 \sum_{i\in S} w_i$ 每条边被两个顶点关联，因此总价格被计算两次

Randomized Algorithms

A Las Vegas randomized algorithm always produces the right answer. But it's running time can vary depending on its random choices.
A Monte Carlo algorithm always has the same running time. But it may produces the wrong answer, depending on its random choices.

Max Cut: Put each node in a random side with probability 1/2. Lemma In a graph with e edges, the algorithm produces a cut with expected size e/2. Proof a) Let X be a random variable equal to the size of the cut. (i.e., Xe=1 if e is in the cut and 0 otherwise) We want to bound E[X]. b) $X = \sum_e X_e$ c) Given an edge e=(i,j), e is in the cut if i and j are on different sides. u 和 v 分属不同子集的概率为$Pr[(u \in A \cap v \in B) + (u \in B \cap v \in A)] = 1/4 + 1/4 = 1/2$, So $E[X_e] = 1/2$. d) $E[X] = \sum_{e\in E} E[X_e] = e \times 1/2 = e/2$

Contention Resolution: n 个进程竞争访问一个共享信道, Each process requests access to the channel at time t with probability p = 1/n.
单个进程的成功概率 Let S[i, t] = event that process i succeeds in accessing the database at time t. By independence, $Pr[S[i,t]]=p\times(1-p)^{n-1}==\frac{1}{n}\times\left(1-\frac{1}{n}\right)^{n-1}$ → $1/en \leq Pr[S[i,t]] \leq 1/2n$ 单个进程的失败概率 Let F[i, t] = event that process i fails to access database in rounds 1 through t. $Pr[F[i,t]] \leq (1-1/e\cdot n)^t$, $Pr[F[i,t]] \leq (1/e)^{c\cdot \ln n} = n^{-c}$ 所有进程的成功概率. Let F[t] = event that at least one of the n processes fails to access database in any of the rounds 1 through t : $F[t] = \bigcup_{i=1}^n F[i,t]$, $Pr[F[t]] \leq \sum_{i=1}^n Pr[F[i,t]]$

Global Mini Cut: Contract edge e.– replace u and v by a single new supernode w –preserve edges, updating endpoints of u and v to w –keep parallel edges, but delete selfloops □ Repeat until graph has just two supernodes v1 and v2 □ Return the cut (between the two supernodes).
Claim. The contraction algorithm returns a min cut with prob $2/n^2$ Proof Consider a global min-cut C*(A*, B*) of G. Let F* be edges with one endpoint in A* and the other in B*. Let k = |F*| = size of min cut. 算法成功的关键：在所有收缩步骤中，始终不收缩 F*中的边，由于 C*是最小割，当前图中每个超级节点的度数至少为 k（否则存在更小的割），因此当前图的总边数$|E'| \geq \frac{1}{2}kn'$（每个边贡献 2 个度数，总度数≥k n'）。收缩 F*中边的概率为 algorithm contracts an edge in F* with probability $|F^*|/|E'| = k/|E'| \leq k/(\frac{1}{2}kn') = 2/n'$
总成功概率（所有 n-2 轮均不收缩）Pr[成功]=$Pr[E_1]\times Pr[E_2|E_1]\times\ldots\times Pr[E_{n-2}|E_1\cap\ldots\cap E_{n-3}]$ $\left(1-\frac{2}{n}\right)\times\geq\left(1-\frac{2}{n-1}\right)\times\ldots\times\left(1-\frac{2}{3}\right)=2/n(n-1)\geq 2/n^2$

- Let (S,t) be an instance of SUBSET-SUM.
  - □ Let s be sum of all elements in S.
- Make a set of jobs J = SU{s-2t}, and schedule them on 2 processors.
- **Claim** If some subset of S sums to t, then min makespan is s-t.
- **Proof** Say S'⊆S sums to t. Schedule the jobs in S' and job s-2t on processor 1. So proc 1 finishes at time t+s-2t=s-t. Proc 2 does the jobs in S-S', so it finishes at time s-t as well.
- **Claim** If the min makespan is s-t, there exists a subset of S that sums to t.
- **Proof** Suppose WLOG proc 1 does the s-2t job. Since makespan is s-t, the other jobs proc 1 does must have total size s-t-(s-2t)=t.
- So (S,t) is yes instance of SUBSET-SUM iff makespan = s-t.
  - □ So SUBSET-SUM ≤ p scheduling, and scheduling is NP-complete.

$$(LP)\ \min \sum_{i\in V} w_i x_i$$
$$\text{s.t.}\quad x_i + x_j \geq 1 \quad (i,j)\in E$$
$$x_i \geq 0 \quad i \in V$$

$$(IP)\ \min\ L$$
$$\text{s.t.}\quad \sum_i x_{ij} = t_j \quad \text{for all } j\in J$$
$$\sum_j x_{ij} \leq L \quad \text{for all } i\in M$$
$$x_{ij} \in \{0, t_j\} \quad \text{for all } j\in J \text{ and } i\in M_j$$
$$x_{ij} = 0 \quad \text{for all } j\in J \text{ and } i\notin M_j$$